

Adora — EOS Integration into 3D Synthesis

As-built notes · Ivan Milic · 2026-09-19 · session summary

Wired the `eos_mlp` EOS (see [eos_mlp/adora_eos_mlp.pdf](#)) into the 3D synth stack: the synth now takes **gas pressure** P_g (or ρ) per depth and the MLP supplies the previously-free (n_e , n_{Htot}). The RT core is *unchanged* — every new line is tagged `# eos`, and the legacy n_e/n_H path still works.

1. Where the EOS is injected (design)

At the **column level**, **once per column**, **above the wavelength** `vmap` — not inside `lte_polarised_rt`. Two reasons: (i) efficiency — the EOS runs once per column and its (n_z ,) outputs are reused across all wavelengths; (ii) **inversion** — the EOS sits *inside* the jitted/traced function, so `jacrev/grad` of the synth gives $\partial \text{Stokes} / \partial P_{\text{gas}}$ straight through the EOS by the chain rule. The fixed MLP is **baked into the compiled function** (closed over) by a small factory — not threaded through `jit` (its params dict has an untraceable string field).

2. `synth3d.py` (kept lean)

Three orthogonal axes — input (n_e, n_H vs EOS P_g/ρ), gradient, call style — factored, instead of writing out their cross-product:

- `KEYS_PG = (T, Pg, vz, vturb, b, gb, cb)` and `KEYS_RHO` — 7-key cube interfaces (P_{gas} or ρ replacing n_e , n_H). `KEYS` (legacy) kept.
- `_finalize(cube_fn, forward_only)` — one shared `jit/stop_gradient` wrapper; `synth_cube` and `synth_cube_fwd` are now one-liners through it, over the single engine `synth_column`.
- `build_synth_cube_eos(eos_params, forward_only=False)` — the EOS factory (a factory, not a module-level `jit`: the params dict has an untraceable string leaf). One new physics line, n_e , $n_H = \text{eos.ne_nhtot_from_pg/rho}(\dots)$ (by `eos_params["mode"]`); reuses `synth_column` and `_finalize`.
- `synth_cube_map(waves, dz, cube, eos_params=None, mode=None, ...)` — **one** dict-cube wrapper: default = legacy n_e/n_H ; setting `mode/eos_params` switches to the EOS path (keys `KEYS_PG/KEYS_RHO`), caching the built fn. `eos_mlp/` is put on `sys.path` (flat sibling imports), then `import eos`.

3. `synth3d_mp.py`

`synth_cube_mp(..., eos_mode=None)`: `None` = legacy n_e/n_H ; `"pg"/"rho"` ship P_g/ρ instead. Because the EOS params are JAX arrays, each worker **builds the EOS forward-synth inside** `_init` (after its JAX import, alongside `read_kurucz`) — never pickled across the spawn boundary — and `_worker` selects cube keys by mode.

4. `muram_loader.py`

`load_muram_cube(..., eos_input="pg"|"rho"|"both"|"none")`: `"pg"` adds cube["Pg"] from MURaM's own EOS pressure (eosP , $\text{dyn/cm}^2 \rightarrow \text{Pa}$; ideal-gas fallback $(n_H \Sigma_A + n_e) k_B T$), `"rho"` adds cube["rho"]. n_e , n_H still emitted; this retires the 1.4 amu n_H cheat for the synth path. P_g in **Pa**, all SI.

5. Validation (FAL-C toy cube)

EOS(P_g) vs the n_e/n_H path: continuum $\sim 2\%$, line I/I_c to $3e-3$ (expected LTE-vs-FAL n_e difference); `jax.grad` w.r.t. P_g *through the EOS* is finite and non-zero; the mp path is **bit-identical** to single-device ($\max|\Delta|=0$). Unchanged after the §2 consolidation.

6. How to run the 3D synthesis (after the update)

The EOS grids + trained MLP already exist in `eos_mlp/` (rebuild once with `python generate_grid.py --workers 16` then `python train_eos.py --mode both` only if the `.npz` are missing). Then, from [3dtesting/](#):

```
from muram_loader import load_muram_cube          # 1. MURaM snapshot -> cube carrying Pg
cube, dz, meta = load_muram_cube(datapath, iteration, kind="muramsub", eos_input="pg")
from synth3d import synth_cube_map                # 2a. GPU, whole cube in one shot
I = synth_cube_map(waves, dz, cube, mode="pg")      # -> (nx, ny, 4, nwave)
from synth3d_mp import synth_cube_mp              # 2b. CPU pool; call under '__main__'
I = synth_cube_mp(waves, dz, cube, nworkers=64, tile=16, eos_mode="pg")
```

Use `mode/eos_mode="rho"` (+ `eos_input="rho"`) for the density head; omit them for the unchanged legacy n_e/n_H path. `waves` is a vacuum grid (e.g. `lw.air-to-vac`); large cubes still need column batching (GPU) or a smaller `tile` (CPU).

7. What's next

Wire the `pg` head into the **inversion**: per-depth free vector (T , P_g , v_z , v_{turb} , b , g_b , cb) (n_e , n_H no longer independent), `response = jacrev(build_synth_cube_eos)` w.r.t. (T , P_g , ...) — $\partial I / \partial P_{\text{gas}}$ for free. Optional: auto-batched GPU `synth_cube_map`, a `rho`-head demo. Files uncommitted.